STAT 432 Homework (Week 10): Random Forests

Neel Nigam

November 01, 2026

All forests are fit with the `RLT` package (version 6.1.1).

Question 1: Bias-variance trade-off simulation

Random seed: 432. `set.seed(432)` controls the data generation, and each RLT fit gets a seed drawn from that stream, so the whole simulation is reproducible.

Setup:

- $X \sim N_5(0, \Sigma)$ with unit variances and pairwise correlation 0.3. $Y = X_1 + X_3 + X_5 + \varepsilon$ with $\varepsilon \sim N(0, 1)$.
- 200 test points are generated once and held fixed. Each of the 100 repetitions draws 200 new training observations, and every tuning setting in that repetition uses the same training set.
- Each forest has 500 trees, and each tree draws 200 observations with replacement (`resample.replace = TRUE`, `resample.prob = 1`). All other arguments are left at their defaults.
- In RLT the node size is `nmin`.

At test point x_j, with $\hat{f}_r(x_j)$ the prediction from repetition r and $\bar{f}(x_j)$ the average over repetitions,

$$\text{Bias}^2(x_j) = \left(\bar{f}(x_j) - f(x_j)\right)^2, \qquad \text{Var}(x_j) = \frac{1}{R}\sum_{r=1}^{R}\left(\hat{f}_r(x_j) - \bar{f}(x_j)\right)^2.$$

Both quantities are then averaged over the 200 test points.

```
set.seed(432)
p <- 5; n_train <- 200; n_test <- 200; R <- 100
Sigma <- matrix(0.3, p, p); diag(Sigma) <- 1
f_true <- function(x) x[, 1] + x[, 3] + x[, 5]

x_test <- mvrnorm(n_test, rep(0, p), Sigma)
f_test <- f_true(x_test)

nodesizes <- c(1, 5, 10, 20, 30)   # with mtry = 3
mtrys     <- 1:5                    # with node size = 10

fit_pred <- function(x, y, mtry, nmin, size, seed) {
  fit <- RLT(x, y, model = "regression", ntrees = 500,
             mtry = mtry, nmin = nmin,
             resample.replace = TRUE, resample.prob = size / nrow(x),
             seed = seed, ncores = 2)
  predict(fit, x_test, ncores = 2)$Prediction[, 1]
}
```

```r
# predictions: [test point, setting, repetition]
pred_node <- array(NA, c(n_test, length(nodesizes), R))
pred_mtry <- array(NA, c(n_test, length(mtrys), R))
pred_node100 <- array(NA, c(n_test, length(nodesizes), R))   # extra: 100 obs per tree

for (r in 1:R) {
  x <- mvrnorm(n_train, rep(0, p), Sigma)
  y <- f_true(x) + rnorm(n_train)
  seeds <- sample.int(1e6, 15)
  for (k in seq_along(nodesizes)) {
    pred_node[, k, r]    <- fit_pred(x, y, 3, nodesizes[k], 200, seeds[k])
    pred_node100[, k, r] <- fit_pred(x, y, 3, nodesizes[k], 100, seeds[10 + k])
  }
  for (k in seq_along(mtrys))
    pred_mtry[, k, r] <- fit_pred(x, y, mtrys[k], 10, 200, seeds[5 + k])
}

bias_var <- function(pred, label, values) {
  fbar <- apply(pred, c(1, 2), mean)                    # test point x setting
  bias2 <- colMeans((fbar - f_test)^2)
  vars  <- sapply(seq_along(values), function(k)
             mean(rowMeans((pred[, k, ] - fbar[, k])^2)))
  data.frame(setting = values, bias2 = bias2, variance = vars,
             total = bias2 + vars, row.names = NULL,
             check.names = FALSE) |> setNames(c(label, "bias2", "variance", "bias2 + var"))
}

res_node    <- bias_var(pred_node,    "nodesize", nodesizes)
res_mtry    <- bias_var(pred_mtry,    "mtry",     mtrys)
res_node100 <- bias_var(pred_node100, "nodesize", nodesizes)
```

```r
knitr::kable(res_node, digits = 4,
             caption = "Varying node size (mtry = 3, 200 obs per tree)")
```

Table 1: Varying node size (mtry = 3, 200 obs per tree)

nodesize	bias2	variance	bias2 + var
1	0.0907	0.1966	0.2874
5	0.0930	0.1902	0.2832
10	0.1007	0.1790	0.2797
20	0.1361	0.1631	0.2993
30	0.1898	0.1585	0.3483

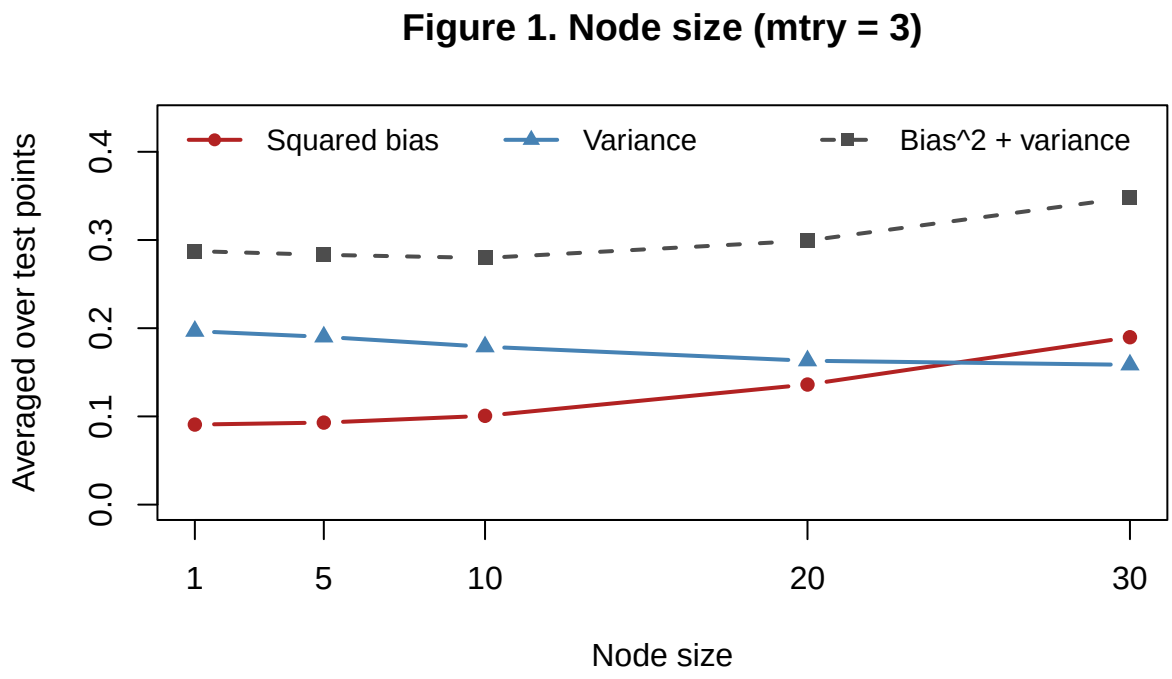
```r
knitr::kable(res_mtry, digits = 4,
             caption = "Varying mtry (node size = 10, 200 obs per tree)")
```

Table 2: Varying mtry (node size = 10, 200 obs per tree)

mtry	bias2	variance	bias2 + var
1	0.3630	0.1098	0.4728
2	0.1489	0.1413	0.2903

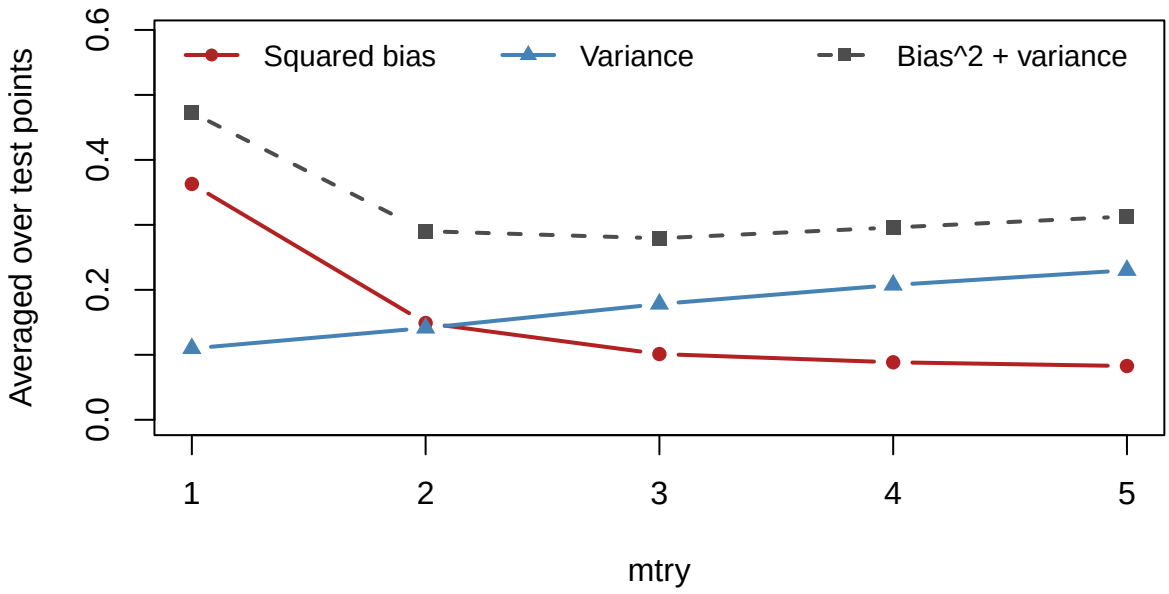
mtry	bias2	variance	bias2 + var
3	0.1012	0.1782	0.2794
4	0.0885	0.2074	0.2959
5	0.0828	0.2301	0.3129

```
bv_plot(res_node, "Node size", "Figure 1. Node size (mtry = 3)")
```




```
bv_plot(res_mtry, "mtry", "Figure 2. mtry (node size = 10)")
```

Figure 2. mtry (node size = 10)



Discussion

Effect of a large node size (30). Increasing node size makes each terminal node average more observations, so the trees are shallower and smoother. In Figure 1 the variance falls steadily as node size grows (from 0.197 at size 1 to 0.158 at size 30), while the squared bias rises sharply (from 0.091 to 0.19). The true function is linear in three predictors, and a forest approximates it with piecewise-constant steps. Large terminal nodes make these steps coarse, so predictions get pulled toward the local mean and the fit underestimates the slope, especially at test points far from the center. With node size 30 the gain in variance is smaller than the added bias, so the total error is the largest of the five settings. Here bias dominates, and the best setting is node size 10.

Benefits and drawbacks of a large mtry. With mtry $= 1$, a split is often forced onto one of the two noise predictors (X_2, X_4). Those splits waste depth, and because the predictors are only moderately correlated with the signal they give a poor approximation of f, so squared bias is high (0.363). As mtry grows, each split is more likely to use X_1, X_3 or X_5, which lowers the bias substantially (0.083 at mtry $= 5$). The drawback is that the trees become more alike. With mtry $= 5$ every tree searches the same variables, and the only randomness left comes from the bootstrap, so the trees are more correlated and averaging removes less variance. Figure 2 shows variance increasing with mtry (0.11 to 0.23). In this example only two of the five predictors are noise, so the reduction in bias outweighs the increase in variance and the total error is lowest at mtry $= 3$. With many irrelevant predictors a small mtry would hurt bias even more, and with many strong but redundant predictors a smaller mtry would be more useful for decorrelating trees.

Sampling 100 observations per tree. I would expect the preferred node size to move toward **smaller** values. Node size limits how many observations a terminal node holds, so with half as many observations per tree the same node size produces trees with roughly half as many leaves. Each tree is therefore coarser and more biased at any fixed node size. To keep the same resolution in x, the node size has to be reduced. Subsampling also makes the trees less correlated, because pairs of trees share fewer observations, so the forest average absorbs more of the variance from deep trees. Both effects favor a smaller node size. The extra simulation below (same training sets, mtry $= 3$, 100 observations drawn with replacement per tree) agrees with this.

```
knitr::kable(res_node100, digits = 4,
             caption = "Extra simulation: varying node size with 100 obs per tree (mtry = 3)")
```

Table 3: Extra simulation: varying node size with 100 obs per tree (mtry = 3)

nodesize	bias2	variance	bias2 + var
1	0.1375	0.1328	0.2703
5	0.1432	0.1280	0.2712
10	0.1651	0.1211	0.2862
20	0.2619	0.1167	0.3786
30	0.4088	0.1206	0.5294

With 100 observations per tree, the best node size is 1 (node size 5 is essentially tied), compared with 10 when each tree uses 200 observations. Every node size has higher squared bias than in the 200-observation setting, and the bias increase from a larger node size is steeper, which pushes the optimum toward small nodes.

Question 2: Tuning random forests for MNIST

```
mnist <- read.csv("mnist-first2000.csv.gz")
digits <- c(1, 6, 7)

train <- mnist[1:1000, ];    train <- train[train$Digit %in% digits, ]
test  <- mnist[1001:2000, ]; test  <- test[test$Digit %in% digits, ]
y_train <- factor(train$Digit, levels = digits)
y_test  <- factor(test$Digit,  levels = digits)
c(train = nrow(train), test = nrow(test))
```

```
train  test
  327   317
```

```
table(y_train)
```

```
y_train
  1   6   7
116  94 117
```

```
# PCA on training pixels: centered, not scaled
pca <- prcomp(train[, -1], center = TRUE, scale. = FALSE)
X_train <- pca$x[, 1:20]
# project test data using training means and loadings
X_test <- scale(as.matrix(test[, -1]), center = pca$center, scale = FALSE) %*%
          pca$rotation[, 1:20]
round(summary(pca)$importance[3, 20], 3)   # cumulative variance explained by 20 PCs
```

```
[1] 0.771
```

The first 20 principal components explain 77.1% of the pixel variance in the training set.

Default forest

Every forest below has 1,000 trees, and each tree draws a bootstrap sample of size n with replacement (`resample.prob = 1`). The RLT defaults are mtry $= \lfloor p/2 \rfloor = 10$ and node size (`nmin`) $= 5$.

```
fit_default <- RLT(X_train, y_train, model = "classification", ntrees = 1000,
                   resample.replace = TRUE, resample.prob = 1,
                   seed = 432, ncores = 2)
fit_default$Error
```

```
[1] 0.04281
```

The default forest has an OOB misclassification rate of **0.0428**, which is 14 of 327 training images.

Tuning grid

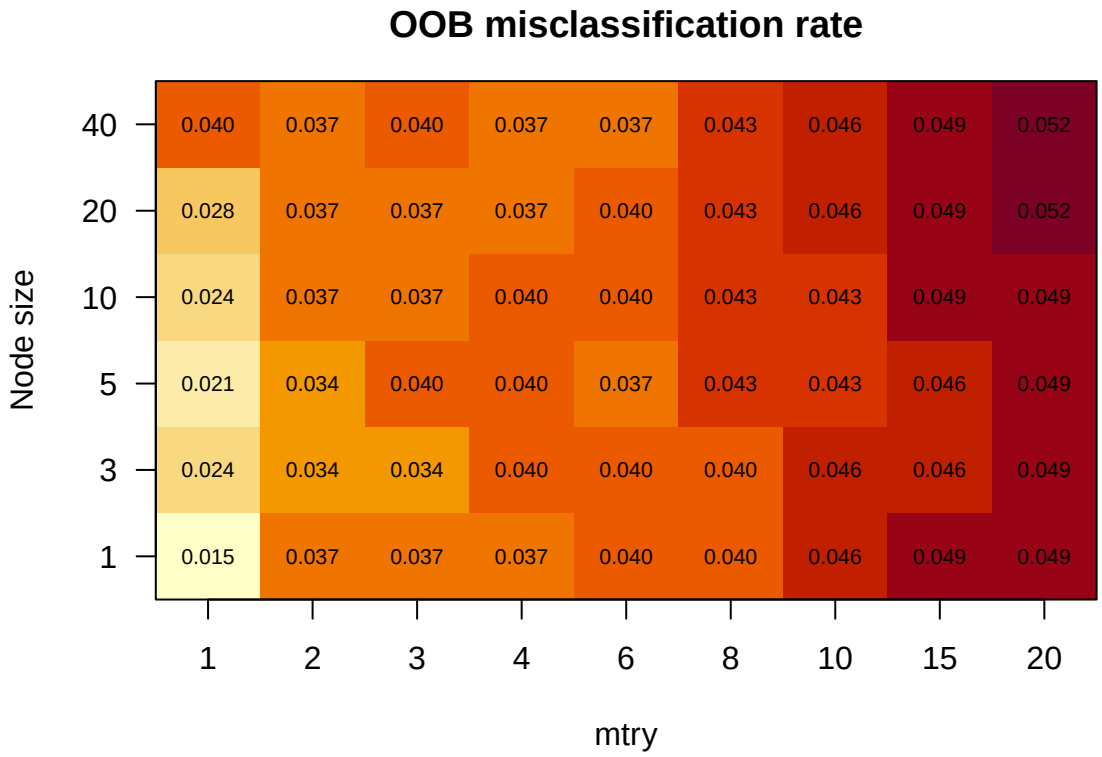
```
grid <- expand.grid(mtry = c(1, 2, 3, 4, 6, 8, 10, 15, 20),
                    nmin = c(1, 3, 5, 10, 20, 40))
grid$oob <- NA
for (i in seq_len(nrow(grid))) {
  fit <- RLT(X_train, y_train, model = "classification", ntrees = 1000,
             mtry = grid$mtry[i], nmin = grid$nmin[i],
             resample.replace = TRUE, resample.prob = 1,
             seed = 432, ncores = 2)
  grid$oob[i] <- fit$Error
}
```

```
oob_tab <- xtabs(oob ~ mtry + nmin, data = grid)
knitr::kable(round(oob_tab, 4),
             caption = "OOB misclassification rate (rows: mtry, columns: node size)")
```

Table 4: OOB misclassification rate (rows: mtry, columns: node size)

	1	3	5	10	20	40
1	0.0153	0.0245	0.0214	0.0245	0.0275	0.0398
2	0.0367	0.0336	0.0336	0.0367	0.0367	0.0367
3	0.0367	0.0336	0.0398	0.0367	0.0367	0.0398
4	0.0367	0.0398	0.0398	0.0398	0.0367	0.0367
6	0.0398	0.0398	0.0367	0.0398	0.0398	0.0367
8	0.0398	0.0398	0.0428	0.0428	0.0428	0.0428
10	0.0459	0.0459	0.0428	0.0428	0.0459	0.0459
15	0.0489	0.0459	0.0459	0.0489	0.0489	0.0489
20	0.0489	0.0489	0.0489	0.0489	0.0520	0.0520

```
par(mar = c(4.5, 4.5, 3, 1))
image(seq_along(unique(grid$mtry)), seq_along(unique(grid$nmin)), unclass(oob_tab),
      col = hcl.colors(30, "YlOrRd", rev = TRUE), axes = FALSE,
      xlab = "mtry", ylab = "Node size", main = "OOB misclassification rate")
axis(1, at = seq_along(unique(grid$mtry)), labels = sort(unique(grid$mtry)))
axis(2, at = seq_along(unique(grid$nmin)), labels = sort(unique(grid$nmin)), las = 1)
text(rep(seq_along(unique(grid$mtry)), times = length(unique(grid$nmin))),
     rep(seq_along(unique(grid$nmin)), each = length(unique(grid$mtry))),
     sprintf("%.3f", as.vector(unclass(oob_tab))), cex = 0.65)
box()
```




```
# lowest OOB error; ties broken toward the simpler model (larger node size, then smaller mtry)
ord <- order(grid$oob, -grid$nmin, grid$mtry)
best <- grid[ord[1], ]
best
```

```
  mtry nmin     oob
1    1    1 0.01529
```

The selected setting is **mtry = 1** and **node size = 1**, with OOB misclassification rate **0.0153**. When settings tie on OOB error, I pick the one with the larger node size and then the smaller mtry, since it is the simpler and less variable forest.

Variable importance of the selected forest

```
fit_best <- RLT(X_train, y_train, model = "classification", ntrees = 1000,
                mtry = best$mtry, nmin = best$nmin,
                resample.replace = TRUE, resample.prob = 1,
                importance = "permute", seed = 432, ncores = 2)
fit_best$Error
```

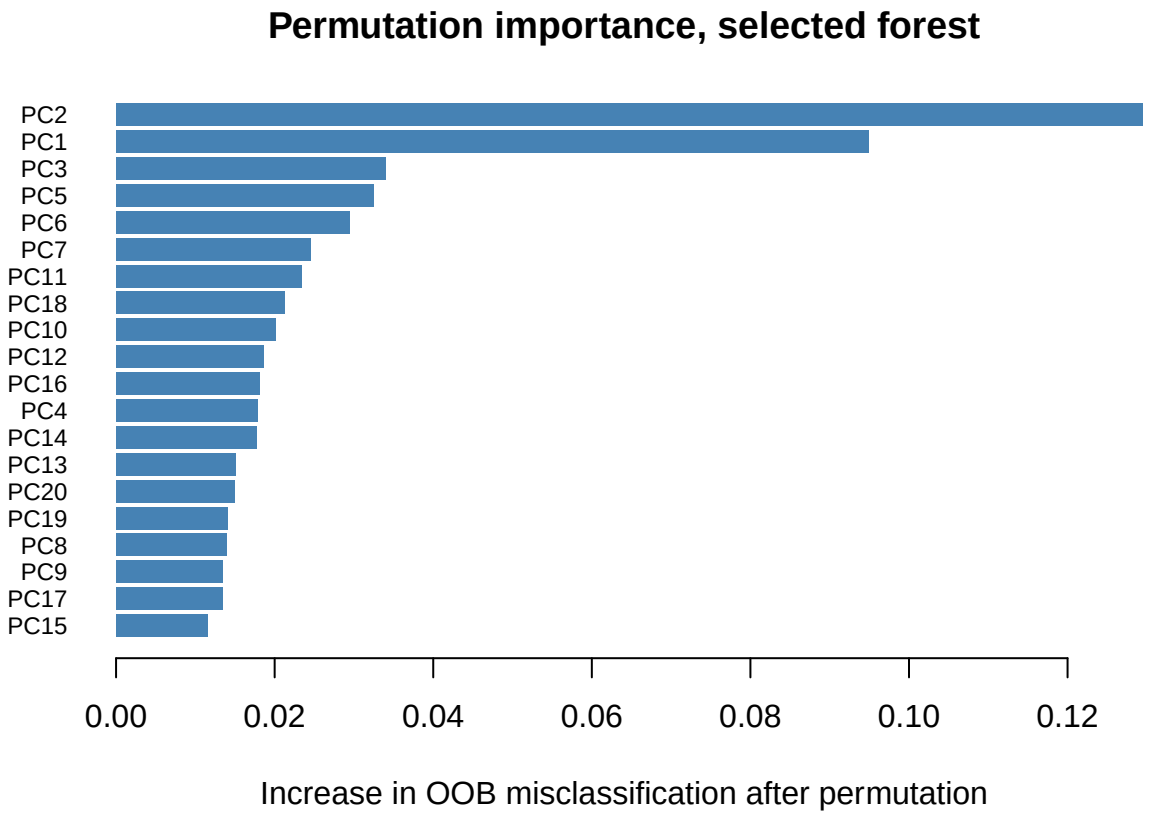
```
[1] 0.01529
```

```
vi <- importance(fit_best)
vi <- vi[order(-vi$VI), ]
knitr::kable(vi, digits = 4, row.names = FALSE,
             caption = "OOB permutation importance (increase in OOB misclassification)")
```

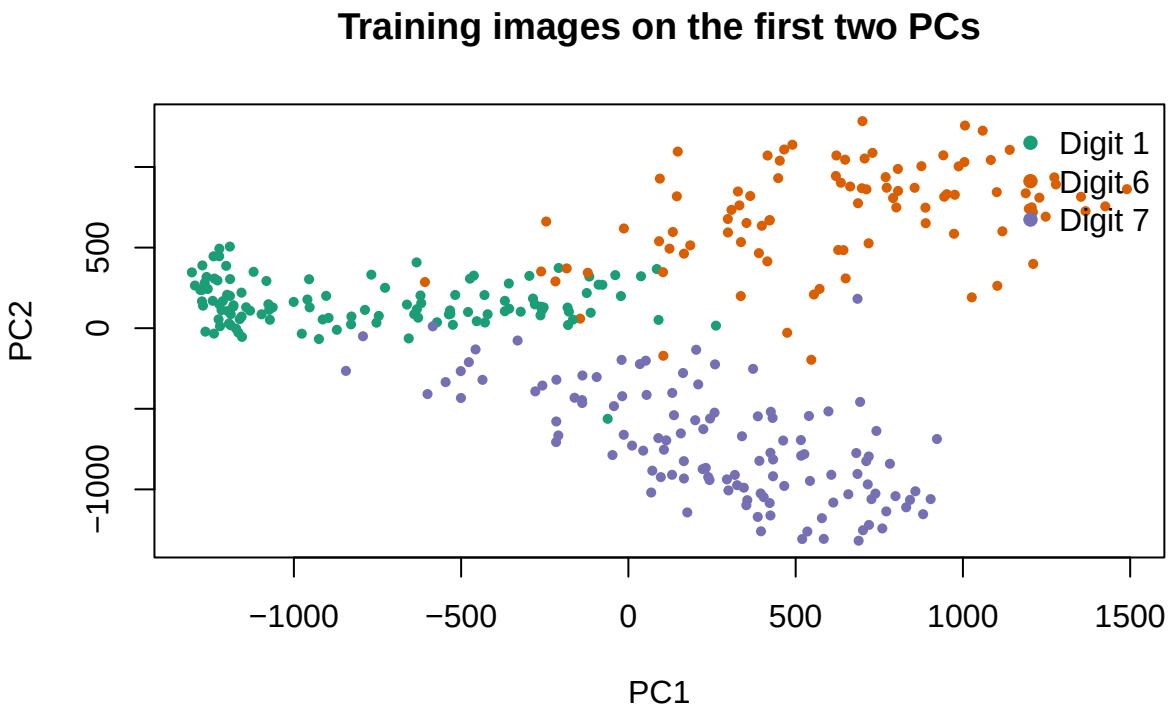
Table 5: OOB permutation importance (increase in OOB misclassification)

Variable	VI
PC2	0.1295
PC1	0.0948
PC3	0.0340
PC5	0.0325
PC6	0.0294
PC7	0.0245
PC11	0.0234
PC18	0.0213
PC10	0.0201
PC12	0.0186
PC16	0.0182
PC4	0.0179
PC14	0.0177
PC13	0.0151
PC20	0.0150
PC19	0.0141
PC8	0.0139
PC9	0.0135
PC17	0.0134
PC15	0.0115

```
par(mar = c(4.5, 4.5, 3, 1))
barplot(rev(vi$VI), names.arg = rev(vi$Variable), horiz = TRUE, las = 1,
        col = "steelblue", border = NA, cex.names = 0.75,
        xlab = "Increase in OOB misclassification after permutation",
        main = "Permutation importance, selected forest")
```




```
cols <- c("1" = "#1b9e77", "6" = "#d95f02", "7" = "#7570b3")
plot(X_train[, 1], X_train[, 2], col = cols[as.character(y_train)], pch = 16, cex = 0.7,
     xlab = "PC1", ylab = "PC2", main = "Training images on the first two PCs")
legend("topright", legend = paste("Digit", digits), col = cols, pch = 16, bty = "n")
```



Comments.

- **mtry drives the error much more than node size.** Each row of the OOB table is fairly flat across node sizes, but the error rises steadily with mtry, from about 0.02 at mtry = 1 to about 0.05 at mtry = 20. The selected forest misclassifies 5 training images out of bag, against 14 for the default (mtry = 10, node size 5).
- **Why a small mtry helps here.** Large mtry lets most trees split on the same strong components (PC1 and PC2), so the trees are highly correlated and averaging removes little variance. With mtry = 1 the split variable is chosen at random, which makes the trees very different from each other. The remaining PCs are not pure noise for this task, since each still carries some class information, so the extra bias from random splits is small and the variance reduction wins. This is the same trade-off as Question 1, but with 20 informative predictors instead of 3 out of 5. Small node sizes work best because classification trees with fully grown leaves are low-bias and the forest average controls their variance.
- **The selection is a little noisy.** One image changes the OOB rate by 1/327, which is about 0.0031. The winner (mtry = 1, node size 1) beats the next best setting (mtry = 1, node size 5) by only 2 images. It also sits on the corner of the grid. Still, the whole mtry = 1 row is better than every other row, so the conclusion that small mtry works best is clear even if the exact node size is not.
- **Importance.** PC2 and PC1 are the most important by a wide margin. The plot of PC1 against PC2 shows the three digits forming fairly separate clouds in these two dimensions, so permuting either one destroys much of the separation. All the other PCs have small but positive importance (roughly 0.01 to 0.03), which fits the explanation above that they carry some class information. Importance does not follow PC order (PC4 ranks below PC11 and PC18, for example), because PCA ranks directions by pixel variance, not by how well they separate the classes. With mtry = 1 the forest is forced to split on every component, which spreads the importance more evenly than a large-mtry forest would. Permutation importance only measures how much the OOB error depends on a PC in this fitted forest. It does not show the direction or shape of the effect.

Test set performance

```
pred_test <- predict(fit_best, X_test)$Prediction
test_err <- mean(pred_test != y_test)
test_err
```

```
[1] 0.01577
```

```
conf <- table(Predicted = pred_test, Actual = y_test)
knitr::kable(conf, caption = "Test confusion matrix (rows: predicted, columns: actual)")
```

Table 6: Test confusion matrix (rows: predicted, columns: actual)

	1	6	7
1	102	0	0
6	0	106	3
7	2	0	104

The selected forest has a test misclassification rate of **0.0158** (5 of 317 test images), which is close to its OOB estimate of 0.0153. The test labels were used only for this final check. The confusion matrix shows where the errors come from. Three 7s were predicted as 6 and two 1s were predicted as 7. Every 6 was classified correctly, and no 1 was confused with a 6. The 1 to 7 errors are expected because both digits are mostly a single slanted stroke. The 7 to 6 errors probably come from 7s written with a curved or closed stem, which look partly like a 6 in the leading PCs. With only 5 errors these patterns are anecdotal, but the overall test error confirms that the OOB estimate was reliable.

Question 3: Planning a random-forest analysis skill

A capable AI already knows how to fit a random forest, tune it by OOB error and report importance, so I would keep the standard part of the skill short. The skill would be a brief checklist that names the required steps and outputs so nothing is skipped, without explaining how to do them. Most of the space would go to things the AI cannot guess, like which package and defaults I want (for example RLT), how I choose between near-tied tuning settings, how data must be split so the test set stays untouched, and how I want tables, figures and written interpretation to look. Detailed procedural instructions can backfire with a strong model because they may override better default judgment or lock in a fixed grid that does not fit the data. So I would state the goal and the constraint, and let the AI fill in the method. For a weaker model I would expand the basic steps into explicit instructions with short code templates, because it is more likely to forget a step such as projecting the test set with training PCA loadings. In both cases I would add one or two reference examples of a finished report, since showing the expected output is often clearer than describing it.

Ten instructions, from basic to personal:

1. Inspect the data first and report the sample size, response type, class balance or response distribution, and any missing values before fitting anything.
2. Hold out the test set at the start, and use it only once for the final assessment. Fit all preprocessing (scaling, PCA, imputation) on the training data only and apply it to the test data.
3. Fit a baseline forest with the package defaults and report its OOB error before any tuning.
4. Tune at least mtry and node size over a grid by OOB error, and report the error for every grid point, not just the winner.
5. Use enough trees for the OOB error to be stable, and check this by comparing OOB error for two different tree counts.
6. Report OOB permutation importance for the selected forest, and add a short note on its limitations (correlated predictors, marginal not causal).

7. Set and report a single random seed, and pass it to the forest function as well as R's RNG, so the full report is reproducible.
8. Use RLT as the forest package, and write the analysis as a knit-ready R Markdown file that produces a PDF.
9. When settings are within one or two observations of the best OOB error, pick the simpler model (larger node size, then smaller mtry) and say that this rule was used.
10. Keep the written interpretation short and direct, with full sentences rather than bullet fragments, few semicolons and em dashes, and every number in the text pulled from the code with inline R rather than typed by hand.